Federated Intrusion Detection Under Label-Flipping Poisoning on CICIoT2023

Salvador Leon Lopez, Elena Victoria Sanchez, Joe Reyna, and April Flores

***Abstract*—Federated learning lets organizations train a shared intrusion detection system (IDS) without pooling raw traffic, but the server cannot inspect client data, so one compromised client can poison the shared model. We study label flipping on CICIoT2023 with ten simulated clients and ask how much detection performance one attacker costs under FedAvg and whether coordinate-wise trimmed-mean aggregation recovers it. This report covers dataset selection, the data pipeline, client partitions, tool choices and the changes made after proposal feedback. A pilot run (12 of 63 files, 5 rounds, one seed) shows that k-means produces degenerate clients, that one attacker costs FedAvg 12.2 macro-F1 points, and that trimmed mean recovers half of that loss.**

***Index Terms*—Federated learning, intrusion detection, data poisoning, label flipping, robust aggregation, CICIoT2023.**

I. Research Question

ORGANIZATIONS that each see different attack traffic could train a shared intrusion detector with federated learning (FL) without pooling raw captures [1], but the server cannot inspect client data, so a compromised participant can poison the shared model [2].

RQ: In a ten-client federated IDS trained on non-IID partitions of CICIoT2023, how much global detection performance is lost when one client performs a label-flipping attack, and does coordinate-wise trimmed-mean aggregation [3] recover it?

Expectation (falsifiable): Under FedAvg, global macro F1 falls by more than 3 points versus the paired clean run, and trimmed mean recovers at least half of that loss. Smaller effects falsify it. Because label flipping need not produce coordinate-wise extreme updates, weak recovery would itself be a meaningful result. A secondary question varies the malicious share (1, 2 and 3 of 10 clients).

II. Dataset Selection

A. Why CICIoT2023

CICIoT2023 [4] provides labeled flows with 39 numeric features (in the merged CSV release) from 33 attacks on a 105-device IoT testbed, grouped by its authors into seven families. It is recent, multi-class and heavily imbalanced, which makes macro F1 and per-family miss rates informative. Its page states no license and asks for citation; we use it for coursework only (accessed Sep. 20, 2026). It is single-testbed lab data, so our clients are *simulated* and all heterogeneity is *simulated statistical heterogeneity*, not evidence about real organizations.

The authors are Group 8 in CSCI 6330: Foundations of Intelligent Security Systems, Department of Computer Science, The University of Texas Rio Grande Valley.

Milestone 1 progress report, submitted October 25, 2026.

TABLE I
Label mapping and counts (pilot: 12 of 63 merged files)

Family	Original labels (count)	Raw rows	Subset
Benign	BenignTraffic (1)	212,961	3,000
DDoS	12 floods and fragmentations	6,586,288	3,000
DoS	UDP, SYN, TCP, HTTP floods (4)	1,567,764	3,000
Recon	4 Recon-* + VulnerabilityScan (5)	133,743	3,000
Web	SQLi, XSS, CmdInj, Upload, Backdoor, BrowserHijack (6)	4,847	3,000
BruteForce	DictionaryBruteForce (1)	2,567	2,565
Spoofing	DNS, MITM-ARP (2)	94,694	3,000
Mirai	greeth, greip, udpplain (3)	511,180	3,000

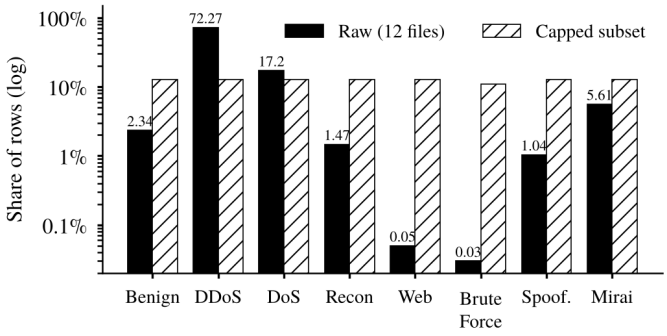


Fig. 1. Family shares before and after capping (pilot, log scale). DDoS falls from 72% of raw rows to 13%; Web and BruteForce rise from under 0.1%.

B. Label Families

Table I gives our mapping, which follows the authors' taxonomy; a unit test checks the family sizes and matching is case-insensitive.

C. Subset and Split Procedure

The order is fixed and scripted (`prepare_data.py`): (1) read all N CSV files (pilot: 12 of 63, 9,114,044 rows) and record label counts *before* sampling; (2) sample at most $\lceil 2 \cdot 45{,}000/N \rceil$ rows per family per file so every capture file contributes; (3) drop rows with inf/NaN (pilot: 1), exact feature+label duplicates (294) and feature vectors with conflicting labels (220); (4) cap each family at 45,000 rows, keeping rarer families whole (pilot cap 3,000: 23,565 rows, split 17,535 / 2,013 / 4,017 across 9 / 1 / 2 files). The subset is therefore closer to balanced than the raw data (Fig. 1); we report both prevalences and never present the subset's as natural; (5) hold out 20% test and 10% validation, stratified by family and

TABLE II
PARTITION CHECKS (PILOT TRAINING POOL, 10 CLIENTS)

	Dirichlet	k-means	IID
Smallest / largest client (rows)	787 / 2,778	1 / 4,804	1,753 / 1,754
Max label purity	0.50	1.00	0.15
Mean TV distance from global	0.43	0.62	0.02
Normalized client–label MI	0.23	0.42	0.00
Checks	PASS	FAIL	PASS

grouped by source file, so no file contributes rows to two splits (the CSVs expose no device or session field, so the file is the finest capture grouping available). Only then are the scaler and partitioners fit, on the 70% training pool. Output hashes are stored in a manifest so every run can be traced to the exact data.

D. Client Partitions

Following the feedback, we use ten clients and a controlled **Dirichlet** label-skew split as the primary non-IID setting [5]: for each family, client shares are drawn from $\mathrm{Dir}(\alpha{=}0.5)$, redrawn until every client holds $\geq$2% of the pool. The label-free k-means partition ($k{=}10$) from the proposal is retained as a reported alternative, and an IID split is the control. Validation and test rows are assigned to clients with the *frozen* train-time partitioner (Dirichlet proportions, or k-means scaler and centroids), never refit, so per-client held-out evaluation is possible. For every client we report size, benign share, family counts, label purity, and total-variation distance from the global distribution, and check: share $\geq$2%, size ratio $\leq$20, purity $\leq$95%, and $\geq$2 families with $\geq$50 rows (Table II). On the pilot, k-means failed every check (a one-row client, label-pure clients, size ratio 4,804), confirming the feedback; Dirichlet passed (Fig. 2).

III. Tool Choices and Method

A. Tools

Python 3.11–3.14, PyTorch, scikit-learn and pandas, with pinned versions. The FL loop is a single-process simulation written by the team (`run_federated.py`), which the project specification permits, instead of Flower [6]: Flower 1.39 deprecated its `run_simulation()` API, and a short loop makes every aggregation step inspectable. FedAvg, trimmed mean and median are unit-tested. The pilot ran on an Apple MacBook Pro, CPU only. All runtimes are single-machine simulation time.

B. Model and Training

One MLP is held constant across all conditions: 39 inputs (signed log transform, then standardization fit on the training pool), hidden layers of 128 and 64 units with ReLU and dropout 0.1, and 8 output logits (13,896 parameters). Loss is unweighted cross-entropy; Adam, learning rate 10^{-3}, batch size 256. The centralized reference trains 30 epochs; FL runs 30 rounds of one local epoch with all ten clients each round and a fresh optimizer state per round. The final epoch/round model is evaluated.

TABLE III
FEEDBACK AND CHANGES

Feedback	Change
k-means clusters may be unequal or label-pure	Dirichlet split is primary; per-client reports and checks for every partition
Use 10–20 clients; vary malicious share	10 clients; 1, 2, 3 attackers
Dedup and grouping before split; freeze partitioner	Duplicates removed first; file-grouped split; frozen held-out assignment
Global and client-level evaluation with support	Per-client held-out metrics; support on every rate
Deterministic attack	Fixed attacker, fixed per-seed poisoned set, labels only
Specify trimmed mean	$\beta{=}0.2$: 2 dropped per side, unweighted mean of 6
Paired factors and seeds	Fixed data seed; identical initial weights per seed; paired deltas
Pilot, mapping, counts, hyperparameters	Pilot configuration; Table I; Section III-B
Communication and hardware	Bytes per client, per round and per run; hardware logged

C. Threat Model and Defense

The attacker is the client with the most attack rows in the frozen training partition (identical for every seed and condition) and must hold $\geq$2 attack families with $\geq$100 rows each. It relabels 50% of its attack rows *within each family* as benign; the poisoned set is drawn once per seed and reused every round. It changes labels only and otherwise behaves honestly. Trimmed mean uses $\beta{=}0.2$: per coordinate, the $\lfloor 0.2 \cdot 10 \rfloor{=}2$ highest and 2 lowest client values are removed and the remaining 6 averaged without size weighting. It is also run without attack, since heterogeneous honest clients can look like outliers.

D. Conditions, Seeds and Metrics

(1) Centralized; (2a/2b) clean FedAvg, IID and non-IID; (3) FedAvg with one attacker; (4a/4b) trimmed mean, clean and attacked; plus 2 and 3 attackers. The data seed (42) fixes records, splits and partitions for all runs. Each run seed (42–44) changes only model initialization, minibatch order and the poisoned draw, so conditions with the same seed start from identical weights and are compared as paired differences; we report individual runs as well as mean and standard deviation. The primary metric is global test macro F1, with per-family precision, recall, FNR and FPR (each with support), binary benign-vs-attack FPR/FNR, PR-AUC, per-client held-out macro F1, and communication: 55.6 kB per client per round in each direction, 1.11 MB per round and 33.4 MB per 30-round run (raw float32, downloads included).

IV. Response to Proposal Feedback

Table III summarizes how each point of the feedback was addressed. The largest design change is the move from six k-means clients to ten Dirichlet clients; k-means remains as an alternative so the effect of the partition can still be discussed.

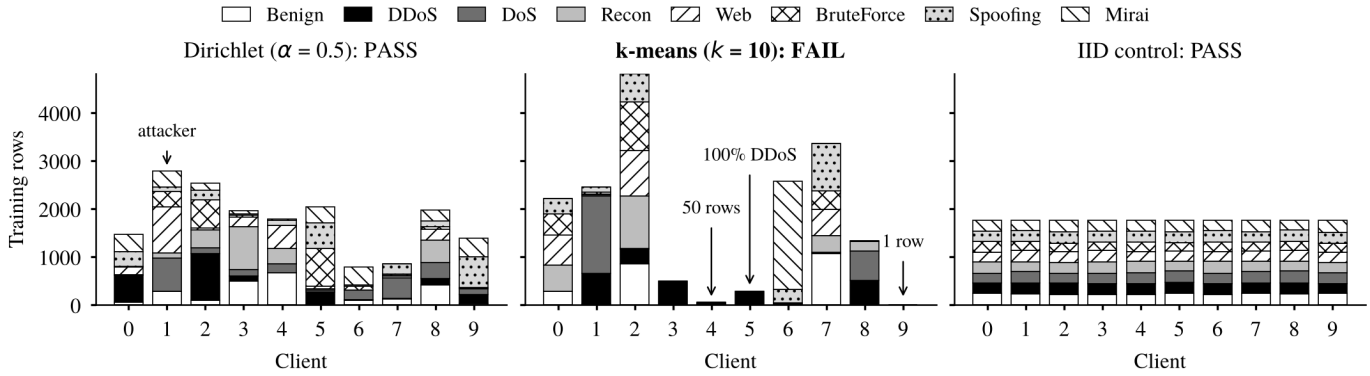


Fig. 2. Training rows per client by attack family for the three partitions (pilot). k-means produces a one-row client, a 50-row client and single-family clients; Dirichlet gives uneven but mixed clients. Client 1, the largest, is the attacker.

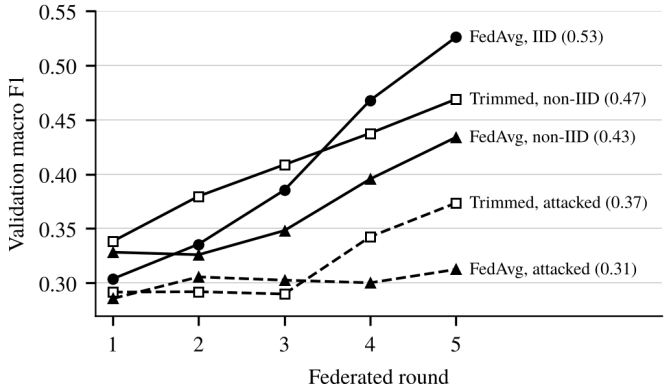


Fig. 3. Pilot validation macro F1 per federated round (seed 42).

TABLE IV
PILOT RESULTS (VALIDATION SET, SEED 42, 5 ROUNDS)

Condition	Macro F1	Binary FNR
Centralized	0.647	0.116
FedAvg, IID	0.526	0.122
FedAvg, non-IID	0.434	0.088
FedAvg, non-IID, 1 attacker	0.312	0.473
Trimmed mean, non-IID	0.469	0.240
Trimmed mean, non-IID, 1 attacker	0.374	0.449

V. STATUS AND NEXT STEPS

Done: the full pipeline (subset, splits, three partitioners with checks, centralized model, FL loop, attack, FedAvg, trimmed mean, median, summary tables) is implemented, and 23 unit and end-to-end tests pass. **Pilot** (Table IV, Fig. 3; 12 files, 3,000 per family, 5 rounds, seed 42; validation macro F1): centralized 0.647; FedAvg IID 0.526, non-IID 0.434. With one attacker (client 1, 1,251 of 2,500 attack rows flipped), FedAvg falls to 0.312 (-12.2 points) and its binary FNR rises from 0.09 to 0.47; trimmed mean reaches 0.469 clean and 0.374 attacked, recovering 6.1 points (50% of the loss). These single-seed, short runs only check that the pipeline behaves as expected; we draw no conclusions from them. **Next (Milestone 2, Nov. 15):** run clean FedAvg on the full data with three seeds, then conditions 3, 4a and 4b and the attacker sweep, reporting paired differences. The test set is used once, after all settings are frozen. **Limitations:** single-testbed lab data; simulated clients; a capped subset whose prevalence differs from the raw data; three seeds support paired comparisons, not distributional claims.

REFERENCES

[1] B. McMahan *et al.*, "Communication-efficient learning of deep networks from decentralized data," *Proc. AISTATS*, 2017.
[2] V. Tolpegin, S. Truex, M. E. Gursoy, and L. Liu, "Data poisoning attacks against federated learning systems," *Proc. ESORICS*, 2020.
[3] D. Yin, Y. Chen, K. Ramchandran, and P. Bartlett, "Byzantine-robust distributed learning," *Proc. ICML*, 2018.
[4] E. C. P. Neto *et al.*, "CICIoT2023: A real-time dataset and benchmark for large-scale attacks in IoT environment," *Sensors*, vol. 23, no. 13, 2023.
[5] T.-M. H. Hsu, H. Qi, and M. Brown, "Measuring the effects of non-identical data distribution for federated visual classification," arXiv:1909.06335, 2019.
[6] D. J. Beutel *et al.*, "Flower: A friendly federated learning framework," arXiv:2007.14390, 2020.